

6th EXERCISE: THE STAIR CLIMBING PROBLEM

WHAT'S THE PROBLEM?

Imagine you're climbing a staircase. You can take 1, 2, or 3 steps at a time.

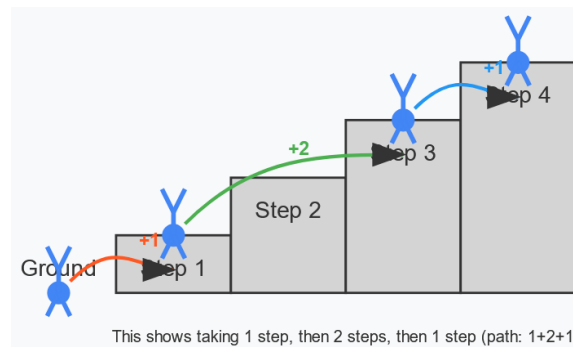
The question is: **how many different ways can you climb a staircase with n steps?**

For example, if you have a staircase with 4 steps, you can climb it in 7 different ways:

1. $1+1+1+1$ (four single steps)
2. $1+1+2$ (two single steps followed by one double step)
3. $1+2+1$ (one single step, one double step, one single step)
4. $2+1+1$ (one double step followed by two single steps)
5. $2+2$ (two double steps)
6. $1+3$ (one single step followed by one triple step)
7. $3+1$ (one triple step followed by one single step)

UNDERSTANDING THE PROBLEM WITH A PICTURE

Let's visualise this problem by drawing a staircase with 4 steps and showing one possible way to climb it:



If you take steps "1+2+1", it means:

First, you climb 1 step (from ground to step 1)

Then, you climb 2 steps (from step 1 to step 3)

Finally, you climb 1 step (from step 3 to step 4)

THE MATHEMATICAL THINKING

To solve this problem, we can think about it step by step:

To reach the n th step, your last move must have been one of:

- A single step from step $n-1$
- A double step from step $n-2$
- A triple step from step $n-3$

Since these are the only possible final moves, we can say that the total number of ways to climb n steps is:

$$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2) + \text{ways}(n-3)$$

This means: "The number of ways to climb n steps equals the number of ways to climb to the step before, plus the number of ways to climb to 2 steps before, plus the number of ways to climb to 3 steps before."

We also need to define what happens at the beginning:

```
ways(0) = 1 (there's one way to climb zero steps: don't climb at all)
ways(n) = 0 for n < 0 (it's impossible to climb a negative number of steps)
```

SOLVING WITH CODE

PYTHON SOLUTION WITH MEMORY STORAGE (MEMOISATION)

```
def climb_stairs(n, memo=None):
    # Create a memory storage if we don't have one yet
    if memo is None:
        memo = {}

    # Base cases
    if n == 0:
        return 1
    if n < 0:
        return 0

    # Check if we've already solved this problem
    if n in memo:
        return memo[n]

    # Recursive case: sum of ways from 1, 2, and 3 steps below
    memo[n] = (climb_stairs(n - 1, memo) +
               climb_stairs(n - 2, memo) +
               climb_stairs(n - 3, memo))

    return memo[n]

print(climb_stairs(4)) # Output: 7
```

This code uses a technique called "memoisation" (storing results we've already calculated) to make it more efficient.

C SOLUTION

```
#include <stdio.h>

#define MAX_STEPS 100
int memo[MAX_STEPS] = {0};

int climb_stairs(int n) {
    // Base cases
    if (n == 0) return 1;
    if (n < 0) return 0;

    // Check if already computed
    if (memo[n] != 0) return memo[n];

    // Recursive case
    memo[n] = climb_stairs(n - 1) + climb_stairs(n - 2) + climb_stairs(n - 3);

    return memo[n];
}

int main() {
    printf("%d\n", climb_stairs(4)); // Output: 7
    return 0;
}
```

SIMPLER APPROACH USING A LOOP

For beginners, a loop might be easier to understand:

```
def climb_stairs_simple(n):
    if n <= 0:
        return 0 if n < 0 else 1
    if n == 1:
        return 1
    if n == 2:
```

```

    return 2

    # Create a list to store the number of ways for each step
    ways = [0] * (n + 1)

    # Set the base cases
    ways[0] = 1
    ways[1] = 1
    ways[2] = 2

    # Calculate the number of ways for each step
    for i in range(3, n + 1):
        ways[i] = ways[i-1] + ways[i-2] + ways[i-3]

    return ways[n]

```

WORKING THROUGH AN EXAMPLE: $n=4$

Let's trace through how our solution works for a 4-step staircase:

1. We need to find $\text{ways}(4)$
 - According to our formula, this equals $\text{ways}(3) + \text{ways}(2) + \text{ways}(1)$
2. Let's find $\text{ways}(3)$:
 - $\text{ways}(3) = \text{ways}(2) + \text{ways}(1) + \text{ways}(0)$
 - We need to find $\text{ways}(2)$ and $\text{ways}(1)$ ($\text{ways}(0) = 1$ by our base case)
3. Let's find $\text{ways}(2)$:
 - $\text{ways}(2) = \text{ways}(1) + \text{ways}(0) + \text{ways}(-1)$
 - $\text{ways}(-1) = 0$ (can't climb negative steps)
 - We need to find $\text{ways}(1)$
4. Let's find $\text{ways}(1)$:
 - $\text{ways}(1) = \text{ways}(0) + \text{ways}(-1) + \text{ways}(-2)$
 - $\text{ways}(0) = 1$, $\text{ways}(-1) = 0$, $\text{ways}(-2) = 0$
 - So $\text{ways}(1) = 1 + 0 + 0 = 1$
5. Now back to $\text{ways}(2)$:
 - $\text{ways}(2) = \text{ways}(1) + \text{ways}(0) + \text{ways}(-1)$
 - $\text{ways}(2) = 1 + 1 + 0 = 2$
6. And back to $\text{ways}(3)$:
 - $\text{ways}(3) = \text{ways}(2) + \text{ways}(1) + \text{ways}(0)$
 - $\text{ways}(3) = 2 + 1 + 1 = 4$
7. Finally, we can compute $\text{ways}(4)$:
 - $\text{ways}(4) = \text{ways}(3) + \text{ways}(2) + \text{ways}(1)$
 - $\text{ways}(4) = 4 + 2 + 1 = 7$

So there are 7 different ways to climb a 4-step staircase!

VISUAL REPRESENTATION OF ALL 7 WAYS

Here are all the possible ways to climb a 4-step staircase:

1. 1+1+1+1: Take one step, then one step, then one step, then one step
2. 1+1+2: Take one step, then one step, then two steps
3. 1+2+1: Take one step, then two steps, then one step
4. 2+1+1: Take two steps, then one step, then one step
5. 2+2: Take two steps, then two more steps
6. 1+3: Take one step, then three steps

7. 3+1: Take three steps, then one step

WHY STORING RESULTS IS IMPORTANT

Without memoisation (storing results), solving this problem for larger staircases would take an enormous amount of time. For instance, calculating the ways to climb a 30-step staircase would require billions of calculations!

With memoisation, we calculate each value only once and remember it, making our solution much faster.

A FUN MATH FACT

The sequence of results for this problem (ways to climb n steps) follows a special pattern called the "Tribonacci sequence": 1, 1, 2, 4, 7, 13, 24, 44, 81...

It's similar to the famous Fibonacci sequence, but each number is the sum of the three previous numbers instead of two.

REAL-LIFE APPLICATIONS

This problem may seem abstract, but similar concepts appear in:

1. **Game Design:** Calculating possible movement patterns in games
2. **Finance:** Analyzing possible sequences of financial events
3. **Probability:** Working out the likelihood of certain sequences of events
4. **Network Routing:** Finding possible paths through communication networks

EXTENDING THE PROBLEM

Once you understand the basic problem, you could make it more challenging:

1. What if you could take 1, 3, or 5 steps at a time? How would the solution change?
2. What if each step size had a different cost, and you wanted to find the cheapest way to climb?
3. What if you could take steps backward as well as forward?

WHAT WE'VE LEARNED

This problem teaches us several important concepts:

1. **Breaking down problems:** Solving complex problems by dividing them into simpler ones
2. **Memoisation:** Saving previous results to avoid recalculating them
3. **Recursive thinking:** Defining a problem in terms of smaller versions of itself
4. **Dynamic programming:** Building solutions by combining solutions to smaller sub-problems

The stair climbing problem is a perfect introduction to these powerful programming techniques.